# Lab 17 – AI for Data Processing

**Name:** Roger A Raju
**Roll.no:** 2503A52370
**Batch:** 13

---

## Task 1 – Customer Data Cleaning

**Task:**

Use AI to generate a Python script for cleaning a customer dataset.

**Prompt:**

*Generate a Python script using Pandas to clean the customer dataset. Handle missing values in age, income, and city, remove duplicate records, convert city names to lowercase, and encode gender and city categorical variables.*

**Code:**

```
"""Task 1: Customer Data Cleaning with Pandas."""

import pandas as pd

data = pd.read_csv("shopping_trends.csv")

print("Original Dataset:")
print(data.head())

data["Age"] = data["Age"].fillna(data["Age"].median())

if "Income" in data.columns:
    data["Income"] = data["Income"].fillna(data["Income"].median())

if "Location" in data.columns:
    data["Location"] = data["Location"].fillna(data["Location"].mode()[0])
    data["Location"] = data["Location"].str.lower().str.strip()

data = data.drop_duplicates()

data["Gender"] = data["Gender"].astype("category").cat.codes

if "Location" in data.columns:
    data["Location"] = data["Location"].astype("category").cat.codes

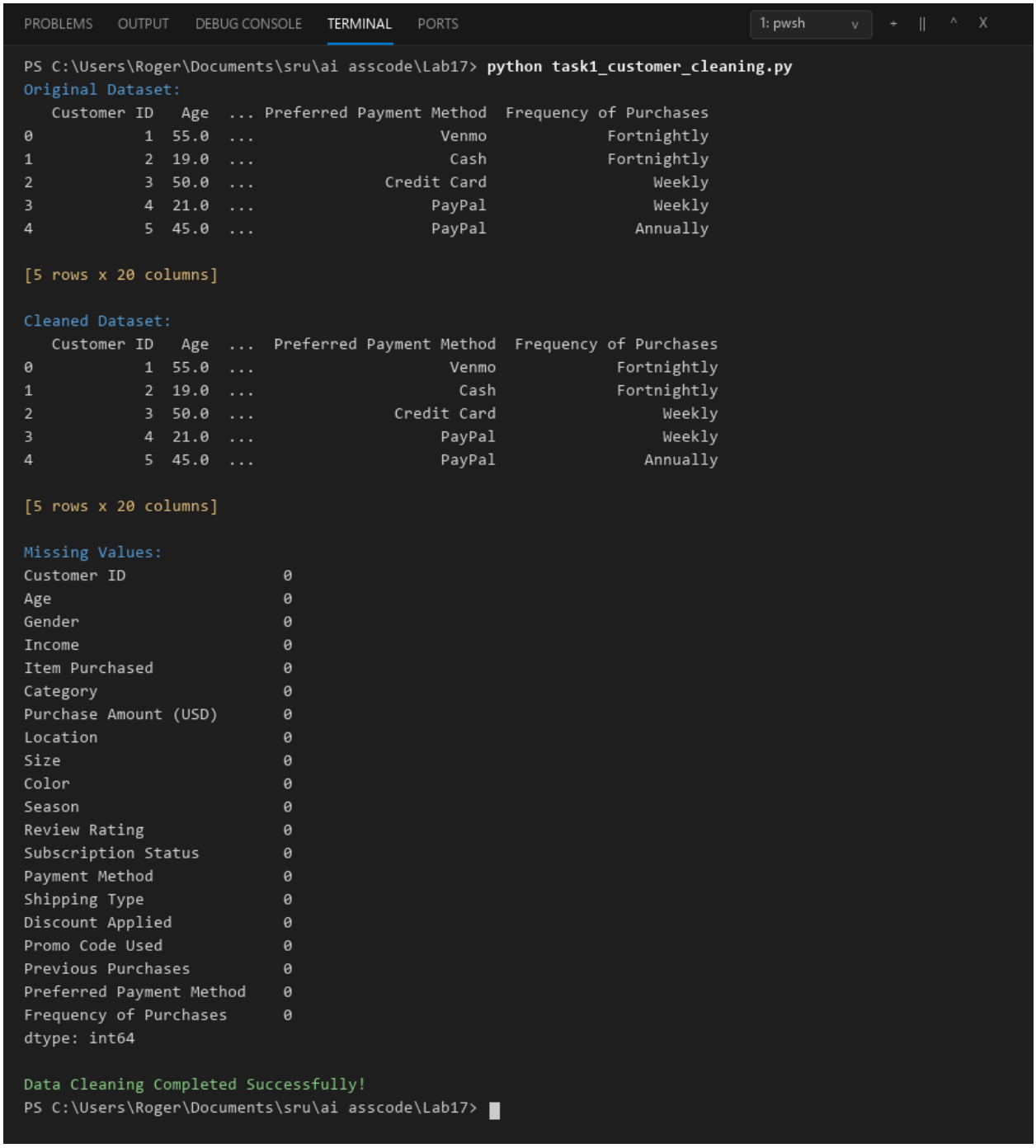
print("\nCleaned Dataset:")
print(data.head())

print("\nMissing Values:")
print(data.isnull().sum())

data.to_csv("cleaned_customer_data.csv", index=False)

print("\nData Cleaning Completed Successfully!")
```

**Output:**




**Justification:**

Cleaned the customer dataset by handling missing values, removing duplicates, standardizing city names, and encoding categorical variables using Pandas.

## Task 2 – Sales Data Preprocessing

**Task:**

Preprocess sales transaction data using AI assistance.

**Prompt:**

*Generate a Python script using Pandas to preprocess sales data. Convert date columns into datetime format, create month, year, and day-of-week features, normalize the sales amount column, and detect and handle outliers.*

**Code:**

```
"""Task 2: Sales Data Preprocessing with Pandas."""

import pandas as pd

data = pd.read_csv("sales_data.csv")

print("Original Dataset:")
print(data.head())

data["Sale_Date"] = pd.to_datetime(data["Sale_Date"])

data["Month"] = data["Sale_Date"].dt.month
data["Year"] = data["Sale_Date"].dt.year
data["Day_of_Week"] = data["Sale_Date"].dt.dayofweek

sales_column = "Sales_Amount"
data[sales_column] = data[sales_column].fillna(data[sales_column].median())

min_value = data[sales_column].min()
max_value = data[sales_column].max()

if max_value != min_value:
    data["Normalized_Sales"] = (data[sales_column] - min_value) / (max_value - min_value)
else:
    data["Normalized_Sales"] = 0

q1 = data[sales_column].quantile(0.25)
q3 = data[sales_column].quantile(0.75)
iqr = q3 - q1

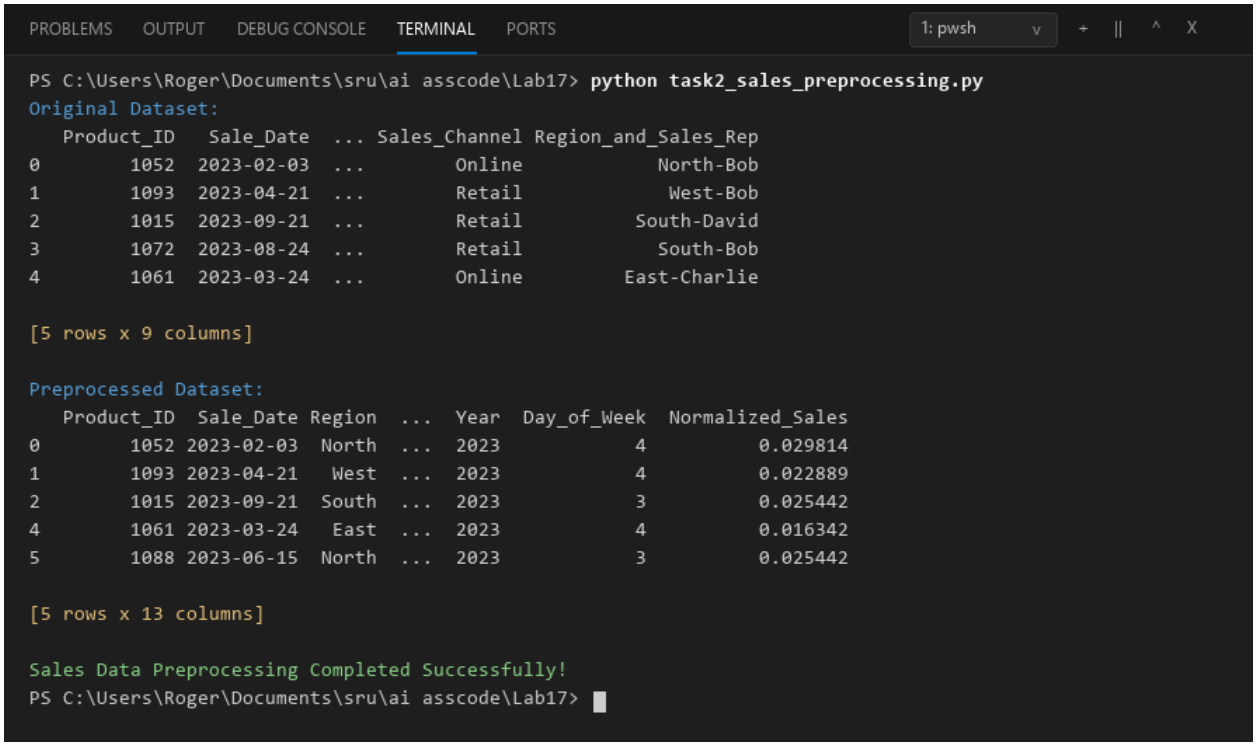
data = data[
    (data[sales_column] >= q1 - 1.5 * iqr)
    & (data[sales_column] <= q3 + 1.5 * iqr)
]

print("\nPreprocessed Dataset:")
print(data.head())

data.to_csv("cleaned_sales_data.csv", index=False)

print("\nSales Data Preprocessing Completed Successfully!")
```

**Output:**




**Justification:**

Preprocessed the sales dataset by converting dates into datetime format, creating month, year, and day-of-week features, normalizing sales amounts, and handling outliers using the IQR method.

## Task 3 – Healthcare Data Preparation

**Task:**

Clean and preprocess a healthcare dataset using AI scripts.

**Prompt:**

*Generate a Python script using Pandas and scikit-learn to prepare the healthcare dataset. Handle missing values in blood_pressure and cholesterol, encode categorical features, scale numerical features using Min-Max scaling, and split the dataset into training and testing sets.*

**Code:**

```
"""Task 3: Healthcare Data Preparation with Pandas and scikit-learn."""

import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split

data = pd.read_csv("dirty_v3_path.csv")

print("Original Dataset:")
print(data.head())

data = data.drop(columns=["random_notes", "noise_col"], errors="ignore")

for col in data.columns:
    if pd.api.types.is_numeric_dtype(data[col]):
        data[col] = data[col].fillna(data[col].median())
    elif not data[col].mode().empty:
        data[col] = data[col].fillna(data[col].mode()[0])

encoder = LabelEncoder()

for col in data.select_dtypes(include=["object", "string"]).columns:
    data[col] = encoder.fit_transform(data[col].astype(str))

print("\nDataset Columns:")
print(data.columns.tolist())

X = data.iloc[:, :-1]
y = data.iloc[:, -1]

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

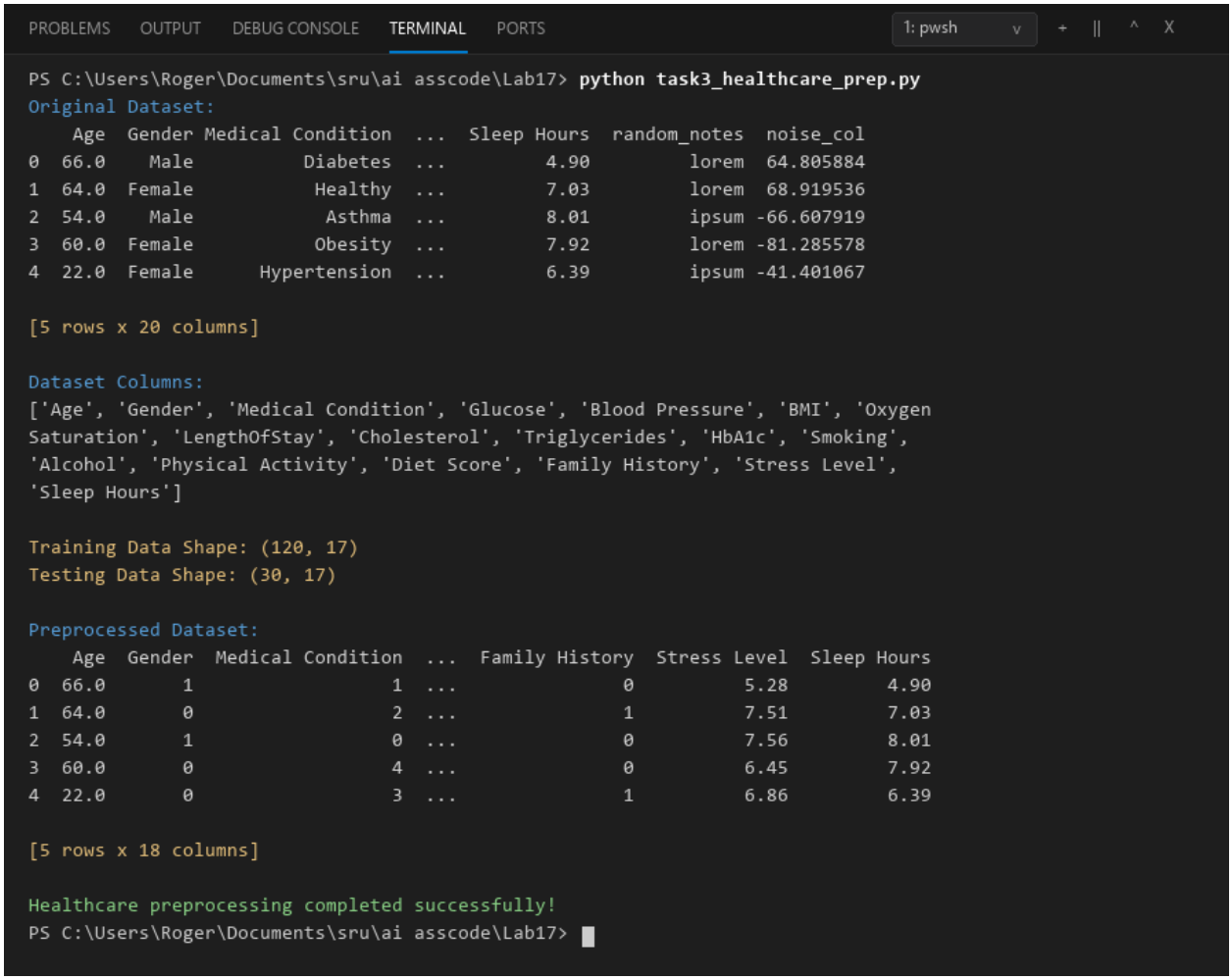
print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

print("\nPreprocessed Dataset:")
print(data.head())

data.to_csv("cleaned_healthcare.csv", index=False)

print("\nHealthcare preprocessing completed successfully!")
```

**Output:**




**Justification:**

Prepared the healthcare dataset by handling missing values, removing unnecessary noise columns, encoding categorical features, scaling numerical data using Min-Max scaling, and splitting the dataset into training and testing sets.

## Task 4 – Employee Salary Data Preprocessing

**Task:**

Clean and preprocess an employee salary dataset using AI scripts.

**Prompt:**

*Generate a Python script using Pandas and scikit-learn to preprocess employee salary data. Handle missing values, detect and remove salary outliers using the IQR method, convert department names to lowercase, and label encode job location and department columns. Display and save the cleaned dataset.*

**Code:**

```
"""Task 4: Employee Salary Data Preprocessing with Pandas and scikit-learn."""

import pandas as pd
from sklearn.preprocessing import LabelEncoder

data = pd.read_csv("Employee_Salary_Dataset.csv")

print("Original Dataset:")
print(data.head())

for col in data.columns:
    if pd.api.types.is_numeric_dtype(data[col]):
        data[col] = data[col].fillna(data[col].median())
    elif not data[col].mode().empty:
        data[col] = data[col].fillna(data[col].mode()[0])

if "Department" in data.columns:
    data["Department"] = data["Department"].str.lower().str.strip()

q1 = data["Salary"].quantile(0.25)
q3 = data["Salary"].quantile(0.75)
iqr = q3 - q1

lower_limit = q1 - 1.5 * iqr
upper_limit = q3 + 1.5 * iqr

data = data[
    (data["Salary"] >= lower_limit)
    & (data["Salary"] <= upper_limit)
]

encoder = LabelEncoder()

for col in ["Gender", "Job_Location", "Department"]:
    if col in data.columns:
        data[col] = encoder.fit_transform(data[col].astype(str))

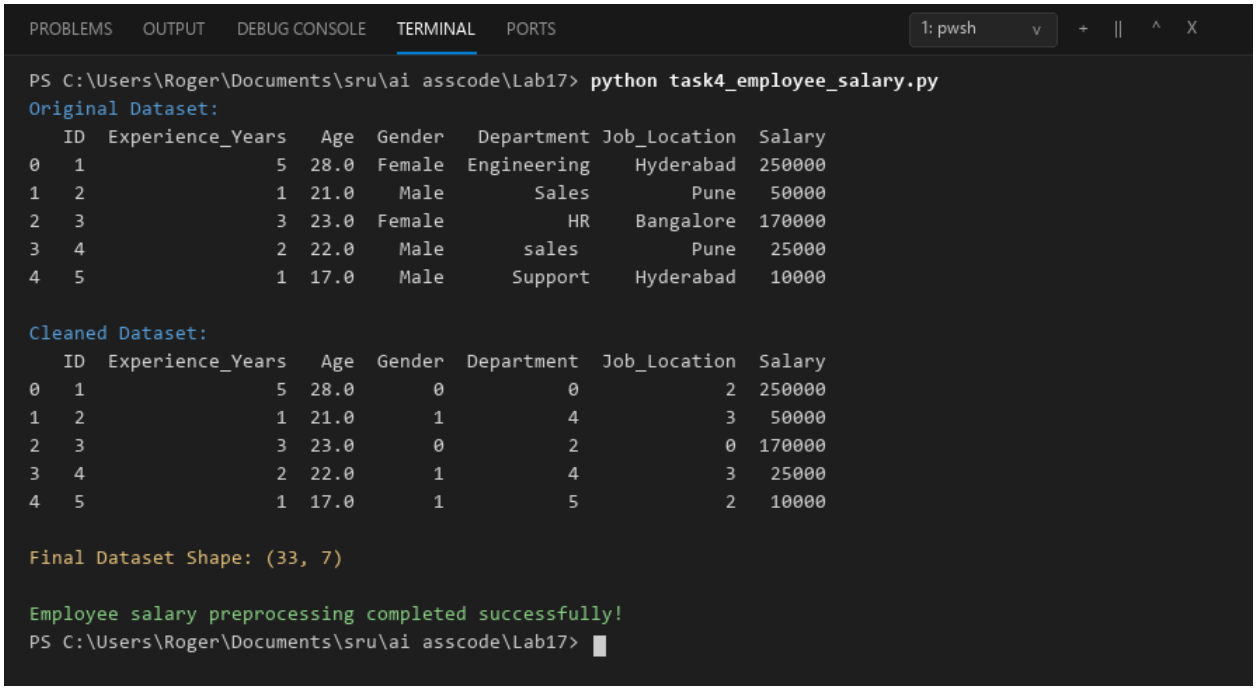
print("\nCleaned Dataset:")
print(data.head())

print("\nFinal Dataset Shape:", data.shape)

data.to_csv("cleaned_employee_salary.csv", index=False)

print("\nEmployee salary preprocessing completed successfully!")
```

**Output:**




**Justification:**

Preprocessed the employee salary dataset by handling missing values, detecting and removing salary outliers using the IQR method, standardizing department names, and encoding categorical features such as gender, job location, and department. Saved the cleaned dataset for further analysis.